

ΚΕΦΑΛΑΙΟ 7

Υλοποίηση εφαρμογών σε προγραμματιστικά περιβάλλοντα

Διδακτικές ενότητες

- 7.1 Προγραμματισμός εφαρμογών για φορητές συσκευές
- 7.2 Αντικειμενοστρεφής προγραμματισμός σε 3D περιβάλλον

Διδακτικοί στόχοι

Σκοπός του κεφαλαίου είναι οι μαθητές να υλοποιήσουν στην πράξη ολοκληρωμένες εφαρμογές σε ένα σύγχρονο περιβάλλον προγραμματισμού, ακολουθώντας βήμα - βήμα όλες τις φάσεις του κύκλου ζωής εφαρμογών.

Οι μαθητές πρέπει να είναι σε θέση:

- ✓ να δημιουργούν μια εφαρμογή με το οπτικό περιβάλλον προγραμματισμού App Inventor για φορητές συσκευές (κινητά, ταμπλέτες-tablets) με λειτουργικό σύστημα Android.
- ✓ να αναγνωρίζουν τις έννοιες κλάση, αντικείμενο, ιδιότητα, μέθοδος και κληρονομικότητα σε ένα αντικειμενοστρεφές περιβάλλον προγραμματισμού.
- ✓ να δημιουργούν έναν εικονικό κόσμο στο τρισδιάστατο (3D) περιβάλλον Alice με δυναμικές κινήσεις χαρακτήρων και αλληλεπίδραση με τον χρήστη.

Ερωτήματα

- ✓ Πώς πιστεύετε ότι μπορείτε να δημιουργήσετε εφαρμογή που θα τη χρησιμοποιείτε στο κινητό σας τηλέφωνο;
- ✓ Γνωρίζετε τις έννοιες *κλάση* και *αντικείμενο* στον αντικειμενοστρεφή προγραμματισμό;
- ✓ Τι είναι η κληρονομικότητα στον αντικειμενοστρεφή προγραμματισμό;
- ✓ Πιστεύετε ότι μπορείτε να δημιουργήσετε ένα παιχνίδι παρόμοιο με το αγαπημένο σας ηλεκτρονικό παιχνίδι;

Βασική ορολογία

Οπτικός προγραμματισμός, αντικειμενοστρεφής προγραμματισμός, κλάση, αντικείμενο, ιδιότητα, κληρονομικότητα, μέθοδος, 3D περιβάλλον, App Inventor, Alice

Εισαγωγή

Το παρόν κεφάλαιο χωρίζεται σε δυο ενότητες. Σε κάθε ενότητα μας δίνεται η ευκαιρία να δημιουργήσουμε μια ολοκληρωμένη εφαρμογή - παιχνίδι εφαρμόζοντας στην πράξη όλες τις φάσεις του κύκλου ζωής μιας εφαρμογής. Τα προγραμματιστικά περιβάλλοντα που αναλύονται ανήκουν στην κατηγορία των ΕΛ/ΛΑΚ (Ελεύθερο Λογισμικό / Λογισμικό Ανοικτού Κώδικα).

7.1 Προγραμματισμός εφαρμογών για φορητές συσκευές

Ανάπτυξη εφαρμογών για φορητές συσκευές

Οι φορητές συσκευές, κυρίως τα έξυπνα κινητά (smartphones) και οι ταμπλέτες (tablets), έχουν διεισδύσει σε πολλούς τομείς της ανθρώπινης δραστηριότητας, όπως είναι η ενημέρωση, η ψυχαγωγία και η εργασία. Οι συσκευές αυτές γίνονται ολοένα και πιο δημοφιλείς και χρηστικές χάρη στο πλήθος εφαρμογών και δυνατοτήτων που διαθέτουν. Επίσης, τείνουν σε αρκετές περιπτώσεις να αντικαταστήσουν τους υπολογιστές και μια πληθώρα άλλων συσκευών, όπως είναι οι φωτογραφικές μηχανές και οι MP3 players.

Οι φορητές συσκευές υποστηρίζονται από Λειτουργικά Συστήματα τα οποία διακρίνονται από συγκεκριμένα χαρακτηριστικά. Τα δημοφιλέστερα Λειτουργικά Συστήματα είναι το **iOS**, το **Android**, το **Windows Phone**, το **Symbian** και το **BlackBerry**. Οι επαγγελματίες προγραμματιστές εφαρμογών για φορητές συσκευές χρησιμοποιούν ολοκληρωμένα περιβάλλοντα ανάπτυξης εφαρμογών, επαγγελματικές γλώσσες προγραμματισμού (π.χ. Java) και αντιμετωπίζουν προβλήματα που σχετίζονται με τους περιορισμένους πόρους των συσκευών (π.χ. επεξεργαστής, μνήμη), με το μικρό μέγεθος της διεπαφής χρήστη, με θέματα ασφαλείας, με τεχνολογίες αυτόματου προσδιορισμού της θέσης του χρήστη κ.ά. Ένας αρχάριος προγραμματιστής που φιλοδοξεί να αναπτύξει τις πρώτες του εφαρμογές για Android μπορεί να χρησιμοποιήσει το εκπαιδευτικό περιβάλλον App Inventor.

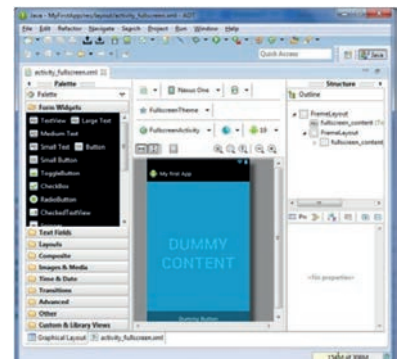
Οι εφαρμογές που αναπτύσσονται για φορητές συσκευές είναι πολλών κατηγοριών: παιχνίδια, ψυχαγωγίας, κοινωνικής δικτύωσης, επικοινωνίας, εκπαιδευτικές, ενημέρωσης, ηλεκτρονικού εμπορίου κ.ά. Οι χρήστες μπορούν να κατεβάσουν τις εφαρμογές της προτίμησής τους, κάποιες δωρεάν και κάποιες άλλες επί πληρωμή, από ηλεκτρονικά καταστήματα, για παράδειγμα το Google Play για το Android, το App Store για το iOS και το Windows Phone Store για το Windows Phone. Οι επαγγελματίες ή ερασιτέχνες προγραμματιστές ανεβάζουν και διαθέτουν τις εφαρμογές τους στα παραπάνω ηλεκτρονικά καταστήματα.

Το εκπαιδευτικό περιβάλλον ανάπτυξης εφαρμογών App Inventor

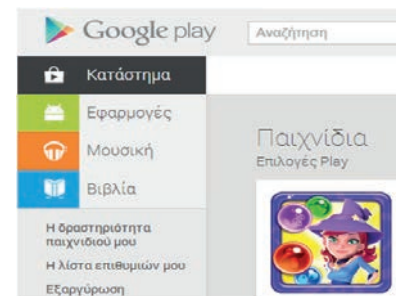
Η ανάγκη για εύκολη ανάπτυξη εφαρμογών για φορητές συσκευές με Android οδήγησε αρχικά τα εργαστήρια της Google στη δημιουργία του App Inventor, ενός ελεύθερου, διαδικτυακού και



Εικόνα 7.1. Δημοφιλή Λειτουργικά Συστήματα για φορητές συσκευές



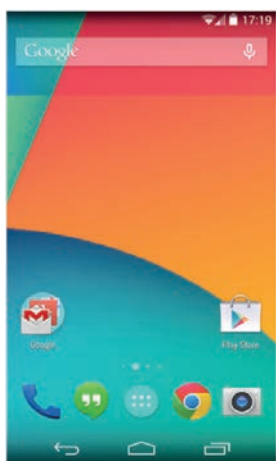
Εικόνα 7.2. Περιβάλλον ανάπτυξης εφαρμογών για φορητές συσκευές με Android



Εικόνα 7.3. Το ηλεκτρονικό κατάστημα Google play



Το **Android** είναι ένα δημοφιλές, ελεύθερο και ανοικτού κώδικα (open source) Λειτουργικό Σύστημα για φορητές συσκευές. Βασίζεται στον πυρήνα του Linux. Το πρώτο κινητό που κυκλοφόρησε με Android έφτασε στα ράφια των καταστημάτων στις 22 Οκτωβρίου 2008.

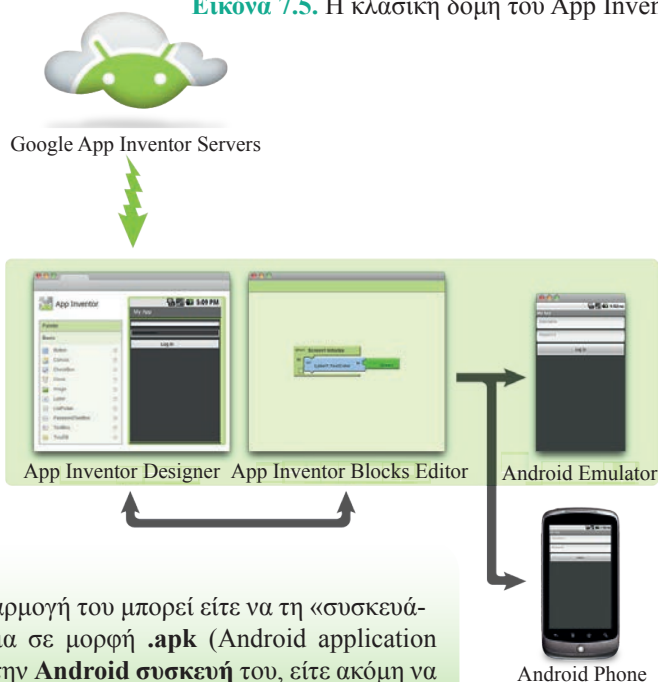


Εικόνα 7.4. Κινητό τηλέφωνο με Android

οπτικού προγραμματιστικού περιβάλλοντος με πλακίδια (blocks), όπως και το Scratch. Στη συνέχεια, το γνωστό κορυφαίο αμερικανικό πανεπιστήμιο MIT (Massachusetts Institute of Technology) ανέλαβε την ανάπτυξη και συντήρηση αυτού. Ακόμα και ένας αρχάριος χρήστης μπορεί να συνδεθεί στο App Inventor και με διαδικασία «σύρε και άφησε» να συνδυάσει πλακίδια και να αναπτύξει εφαρμογές για φορητές συσκευές με Android, το οποίο επίσης κατασκεύασε η Google βασισμένη στο ελεύθερο και ανοικτό λειτουργικό σύστημα για υπολογιστές Linux. Τα πλακίδια ενώνονται μόνο όταν προκύπτει συντακτικά σωστό πρόγραμμα, και η τελική εφαρμογή μπορεί να εκτελεστεί και να δοκιμαστεί είτε απευθείας σε συσκευή που είναι συνδεδεμένη με τον υπολογιστή του χρήστη (ενσύρματα με USB ή ασύρματα με WiFi) είτε σε ενσωματωμένο emulator (προσομοιωτή κινητού τηλεφώνου).

Η κλασική δομή του περιβάλλοντος του App Inventor (εικόνα 7.5) αποτελείται από: (α) τον **Designer** (Σχεδιαστή), όπου ο χρήστης επιλέγει τα συστατικά μέρη για την εφαρμογή που αναπτύσσει, και (β) τον **Blocks Editor** (Συντάκτη πλακιδίων), όπου ο χρήστης συνδυάζει οπτικά τα πλακίδια του προγράμματος, για να ορίσει τη συμπεριφορά των μερών της εφαρμογής (μοιάζει με τη συναρμολόγηση ενός πάζλ). Τα πλακίδια είναι ταξινομημένα σε διαφορετικά χρώματα ανάλογα με τη λειτουργία που επιτελούν.

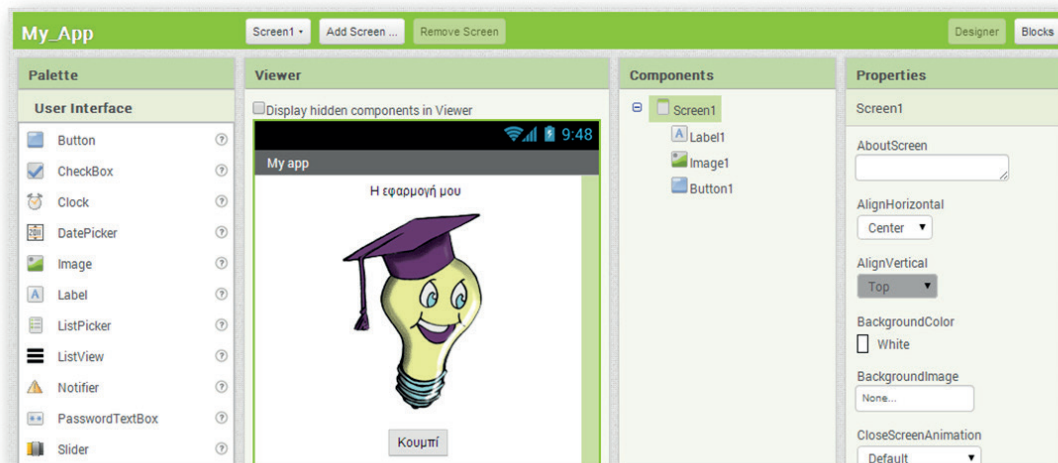
Εικόνα 7.5. Η κλασική δομή του App Inventor



Όταν ο χρήστης ολοκληρώσει την εφαρμογή του μπορεί είτε να τη «συσκευάσει», για να παραγάγει το τελικό πρόγραμμα σε μορφή **.apk** (Android application package), προκειμένου να το εγκαταστήσει στην **Android συσκευή** του, είτε ακόμη να το διανείμει δωρεάν ή εμπορικά στο Google Play. Εναλλακτικά, αν δεν υπάρχει διαθέσιμη κάποια συσκευή Android, ο χρήστης έχει τη δυνατότητα να δημιουργήσει και να ελέγξει τη λειτουργία της εφαρμογής του, χρησιμοποιώντας τον προσομοιωτή **Android Emulator**, ο οποίος είναι λογισμικό που εκτελείται τοπικά στον υπολογιστή του και συμπεριφέρεται σαν ένα κινητό τηλέφωνο.

Διαδικασία δημιουργίας μιας εφαρμογής στο App Inventor

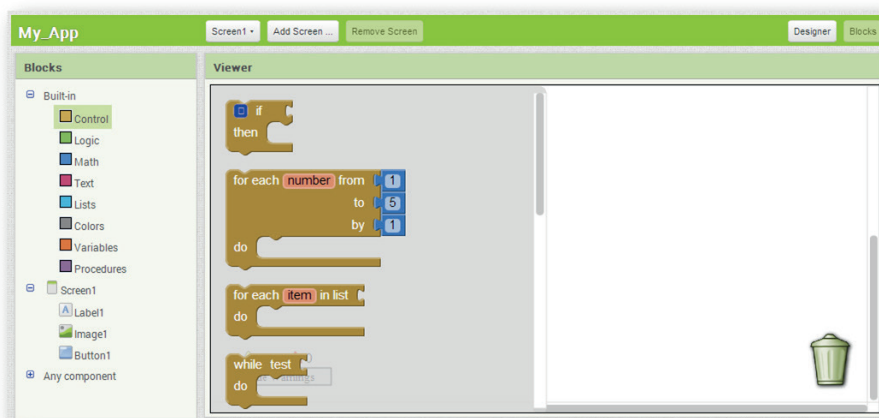
1. Αρχικά, επισκεπτόμαστε τον επίσημο ιστότοπο του App Inventor, ο οποίος περιέχει υλικό στην αγγλική γλώσσα με υποστηρικτικές οδηγίες, οδηγούς εκμάθησης, βιβλιοθήκες, ομάδες συζητήσεων κ.ά.
2. Για να έχουμε δικαίωμα πρόσβασης στο προγραμματιστικό περιβάλλον, θα πρέπει να διαθέτουμε λογαριασμό στην Google. Για όσους δεν έχουν λογαριασμό, η εγγραφή είναι εύκολη και δωρεάν. Επιλέγουμε τον σύνδεσμο **Create** και στο παράθυρο που μας ανοίγει κάνουμε είσοδο με τα στοιχεία του λογαριασμού μας.
3. Στην αρχική οθόνη που εμφανίζεται επιλέγουμε **New Project** (νέο έργο), οπότε και μας ζητείται να δώσουμε ένα όνομα για την εφαρμογή που πρόκειται να δημιουργήσουμε.
4. Ανοίγει η καρτέλα **Designer** (εικόνα 7.6), για να σχεδιάσουμε την εμφάνιση της εφαρμογής μας επιλέγοντας τα απαραίτητα συστατικά στοιχεία και ορίζοντας ιδιότητες γι' αυτά.



Εικόνα 7.6. App Inventor Designer

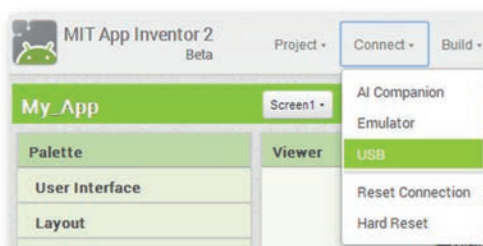
Ο Designer αποτελείται από τα παρακάτω κύρια πλαίσια:

- **Palette (συλλογή συστατικών):** περιέχει όλα τα στοιχεία, χωρισμένα σε κατηγορίες (User Interface, Layout, Media κ.ά.) που μπορούμε να εισάγουμε στην εφαρμογή μας με απλό σύρσιμο.
 - **Viewer (οθόνη συσκευής):** εδώ τοποθετούμε στη θέση που θέλουμε τα συστατικά στοιχεία της εφαρμογής με τη διαδικασία «σύρε και άφησε» από το πλαίσιο Palette.
 - **Components (επιλεγμένα συστατικά):** μια δένδροειδής δομή των στοιχείων που έχουμε επιλέξει.
 - **Properties (ιδιότητες):** το πλαίσιο παραμετροποίησης του κάθε συστατικού (π.χ. χρώμα, μέγεθος, συμπεριφορά).
5. Μόλις ολοκληρώσουμε τη σχεδίαση της εφαρμογής μας και την παραμετροποίηση των συστατικών της μέσω των ιδιοτήτων τους, ανοίγουμε την καρτέλα **Blocks** (εικόνα 7.7). Ο προγραμματισμός γίνεται στο πλαίσιο **Viewer**, όπου σύρουμε από το πλαίσιο **Blocks** τα κατάλληλα πλακίδια και τα συνδυάζουμε, για να ορίσουμε τις συμπεριφορές και τις συσχετίσεις της εφαρμογής μας. Τα πλακίδια είναι χρωματιστά και χωρίζονται σε δύο κατηγορίες: τα ενσωματωμένα (Built-in), που ορίζουν γενικές συμπεριφορές στην εφαρμογή μας, και τα σχετικά με συγκεκριμένα συστατικά της εφαρμογής που ορίζουν συμπεριφορές γι' αυτά.



Εικόνα 7.7. App Inventor Blocks Editor

6. Στο τελευταίο βήμα εγκαθιστούμε και ελέγχουμε την εφαρμογή μας με σύνδεση σε κάποια φορητή συσκευή (εικόνα 7.8). Επιλέγουμε από το μενού **Connect: AI Companion** για σύνδεση μέσω WiFi, με την απαραίτητη προϋπόθεση να το έχουμε πρώτα εγκαταστήσει στη συσκευή μας ή **USB** για ενσύρματη σύνδεση ή **Emulator** για προσομοίωση φορητής συσκευής στον υπολογιστή μας (εικόνα 7.9).



Εικόνα 7.8. Σύνδεση φορητής συσκευής



Εικόνα 7.9. Ο Emulator

Project με την εφαρμογή App Inventor

Στη συνέχεια θα δημιουργήσουμε μια ολοκληρωμένη εφαρμογή ακολουθώντας τις φάσεις του κύκλου ζωής εφαρμογών, όπως τις μελετήσαμε στο κεφάλαιο 5. Υποθέτουμε ότι εργαζόμαστε στην εταιρεία «ΛΑΜΔΑ Software Production» που παράγει προγράμματα και εφαρμογές σε διάφορες γλώσσες προγραμματισμού.

Φάση 1η: Ανάλυση

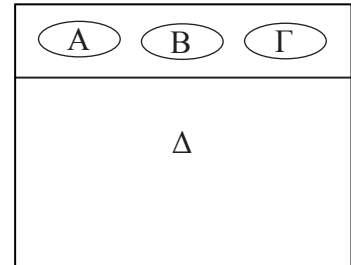
Ένας πελάτης της εταιρείας μάς ζητάει να φτιάξουμε μια εφαρμογή για φορητές συσκευές που λειτουργούν με λειτουργικό σύστημα Android. Η εφαρμογή απαιτείται να είναι πρωτότυπη, για να προκαλέσει το ενδιαφέρον των εφήβων-μαθητών στους οποίους απευθύνεται. Προτιμάμε να γίνει η υλοποίησή της με το περιβάλλον App Inventor.

Ζητείται η εφαρμογή να έχει ένα κεντρικό μενού με 3 επιλογές. Η πρώτη επιλογή να ξεκινάει την εκτέλεση προστασίας της οθόνης, η δεύτερη επιλογή να περιέχει την εκτέλεση ενός παιχνιδιού και η τελευταία επιλογή να υπολογίζει τον Μέσο Όρο ενός μαθήματος.

Φάση 2η: Σχεδίαση

Η ομάδα των προγραμματιστών της εταιρείας συνεδριάζει και καταλήγει στην παρακάτω πρόταση προς τον πελάτη.

Στο σχήμα 7.1 παρουσιάζεται το σχέδιο της διεπαφής χρήσης του κινητού. Τα Α, Β, Γ είναι τα κουμπιά για τις 3 επιλογές και το Δ είναι ο χώρος, όπου θα εμφανίζονται τα αποτελέσματα από την εκτέλεση του προγράμματος που αντιστοιχεί σε κάθε επιλογή. Συγκεκριμένα:



Σχήμα 7.1. Σχεδίαση οθόνης κινητού

- ✓ Όταν πατηθεί το κουμπί Α, εκτελείται η προστασία της οθόνης, όπου εμφανίζεται μια εικόνα ενός ήρεμου σκύλου, και, όταν ο χρήστης αγγίζει την περιοχή (Δ), τότε αλλάζει η εικόνα του σκύλου σε αγριεμένο και ακούγεται ο ανάλογος ήχος.
- ✓ Όταν πατηθεί το κουμπί Β, εκτελείται το παιχνίδι. Ο χρήστης θα έχει τη δυνατότητα να ζωγραφίζει στην οθόνη του κινητού του.
- ✓ Όταν πατηθεί το κουμπί Γ, υπολογίζεται ο Μέσος Όρος του μαθήματος και εμφανίζεται η προαγωγή ή απόρριψη του μαθητή στο συγκεκριμένο μάθημα.

Αναλυτικότερα, η ομάδα σχεδίασε τα παρακάτω πλαίσια (σενάρια εντολών), ώστε στη συνέχεια να τα υλοποιήσει στην επόμενη φάση.

(Α) Για την πρώτη επιλογή προστασίας της οθόνης έχουμε τα παρακάτω σενάρια-ψευδοκώδικα:

Όταν το κουμπί Α πατηθεί, τότε
απόκρυψε ό,τι δεν αφορά στον σκύλο και
εμφάνισε τον ήρεμο σκύλο.

Όταν ο χρήστης αγγίζει την περιοχή Δ, τότε
άλλαξε την εικόνα του σκύλου σε αγριεμένο
και παίξε ήχο γαβγίσματος

Όταν ο χρήστης πάψει να αγγίζει την περιοχή Δ, τότε
άλλαξε την εικόνα του σκύλου σε ήρεμο.

(Β) Για τη δεύτερη επιλογή του παιχνιδιού σχεδίασης έχουμε τα παρακάτω σενάρια-ψευδοκώδικα:

Όταν το κουμπί Β πατηθεί, τότε
απόκρυψε ό,τι δεν αφορά στη ζωγραφική
και καθάρισε την περιοχή Δ

Όταν ο χρήστης αγγίζει την οθόνη, τότε
ζωγράφησε σε εκείνη τη θέση μια τελεία

Όταν ο χρήστης κινήσει το δάκτυλο του
επάνω στην οθόνη, τότε
ζωγράφησε μια γραμμή μεταξύ της τρέχουσας
θέσης του δακτύλου και της προηγούμενης.

Για να μπορέσουμε όμως να σχεδιάσουμε κάτι άλλο από την αρχή, θα πρέπει να καθαρίσουμε την οθόνη.

Όταν κουνηθεί η φορητή συσκευή,
τότε
καθάρισε την οθόνη.

(Γ) Για την τρίτη επιλογή όπου υπολογίζουμε τον μέσο όρο ενός μαθήματος και εμφανίζεται στην οθόνη το αποτέλεσμα της προαγωγής του μαθητή, έχουμε τα παρακάτω σενάρια:

Ορίζουμε και αρχικοποιούμε τις μεταβλητές:

- Α (ο προφορικός βαθμός του Α τετραμήνου σε ένα μάθημα) $\leftarrow 0$ (μηδέν)
- Β (ο προφορικός βαθμός του Β τετραμήνου στο ίδιο μάθημα) $\leftarrow 0$ (μηδέν)
- Γ (ο γραπτός βαθμός στο ίδιο μάθημα) $\leftarrow 0$ (μηδέν)
- ΜΟ (ο μέσος όρος βαθμολογίας του μαθήματος) $\leftarrow 0$ (μηδέν)

Επιλέγουμε να γίνουν οι υπολογισμοί και η εμφάνιση των αποτελεσμάτων με τη χρήση διαδικασιών.

Όταν το κουμπί Γ πατηθεί, τότε
απόκρυψε το σκύλο,
καθάρισε την περιοχή Δ
εμφάνισε την ετικέτα οδηγιών και το πλαίσιο
όπου θα εισάγω τους βαθμούς
εμφάνισε το κουμπί «Τελικό αποτέλεσμα»

Όταν πατήσω το κουμπί «Τελικό αποτέλεσμα»,
κάλεσε τη διαδικασία για τον υπολογισμό του
μέσου όρου
κάλεσε τη διαδικασία για την εμφάνιση των
αποτελεσμάτων

Διαδικασία: Υπολογισμός του μέσου όρου του μαθήματος

Υπολόγισε τον μέσο όρο των 2 προφορικών βαθμών $((A+B)/2)$ και καταχώρισέ τον στον ΜΟ.
Υπολόγισε τον μέσο όρο $((ΜΟ+Γ)/2)$ και καταχώρισέ τον στον ΜΟ.

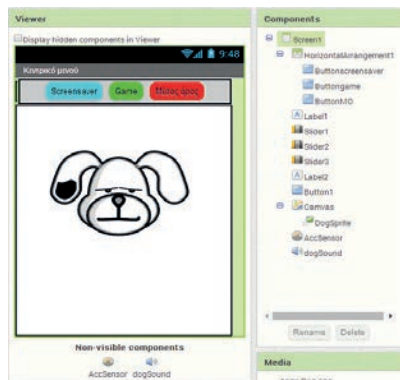
Διαδικασία: Εμφάνιση των αποτελεσμάτων

Εμφάνισε τον μέσο όρο του μαθήματος.
Αν ο ΜΟ είναι μεγαλύτερος ή ίσος από 10, τότε
εμφάνισε ότι ο μαθητής πέρασε το μάθημα
αλλιώς
εμφάνισε ότι ο μαθητής δεν πέρασε το μάθημα.
Τέλος_αν

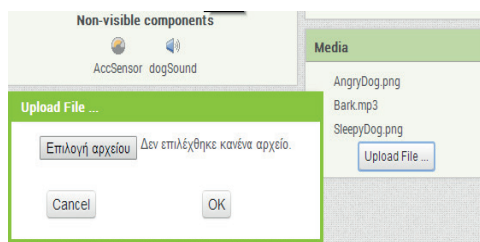
Φάση 3η: Υλοποίηση

Ακολουθούμε τα βήματα δημιουργίας μιας εφαρμογής, όπως περιγράφονται σε προηγούμενη παράγραφο, και δημιουργούμε ένα νέο project με όνομα «Fun & Learn». Βρισκόμαστε στο περιβάλλον εργασίας Designer (σχεδίασης) του App Inventor. Απ' όλη την παραπάνω περιγραφή καταλαβαίνουμε ότι θα χρησιμοποιήσουμε 2 εξωτερικά αρχεία εικόνων του σκύλου και ένα ήχου (γάβγισμα), οπότε, χρησιμοποιώντας το κουμπί Upload File του πλαισίου Media (Εικόνα 7.10), ανεβάζουμε τα σχετικά αρχεία (Προσοχή: το συνολικό μέγεθος των αρχείων δεν πρέπει να υπερβαίνει τα 5 MB, διότι τότε δεν δημιουργείται εκτελέσιμο αρχείο .apk).

Στη συνέχεια εισάγουμε τα παρακάτω στοιχεία στο αντικείμενο Screen1 του πλαισίου Viewer. Αλλάζουμε τις ρυθμίσεις για κάθε αντικείμενο όπως στον πίνακα 7.1. Η τελική μορφή μετά από αυτή την ενέργεια θα πρέπει να είναι αυτή που φαίνεται στην εικόνα 7.11.



Εικόνα 7.11. Τα πλαίσια Viewer και Components μετά την εισαγωγή των αντικειμένων

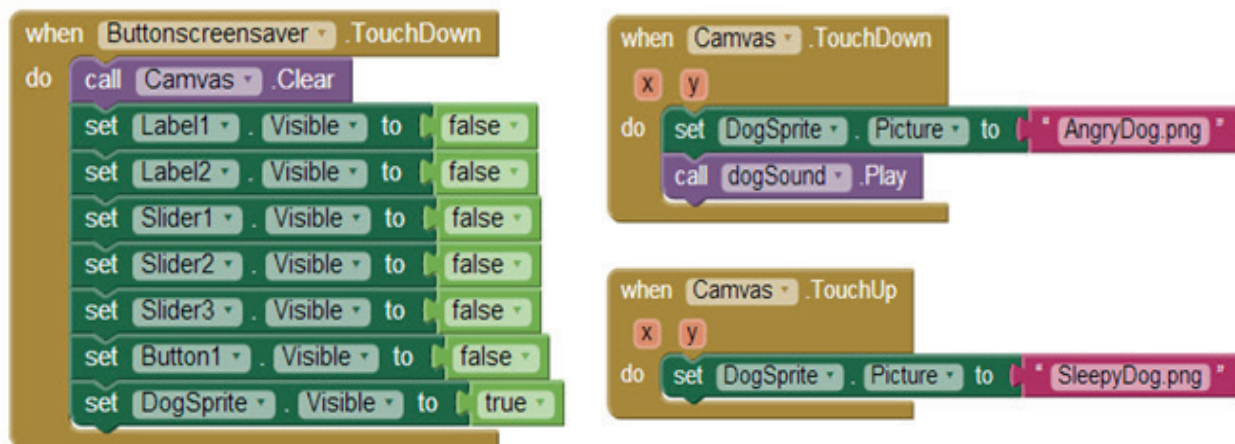


Εικόνα 7.10. Ανέβασμα εξωτερικών αρχείων που θα χρησιμοποιηθούν

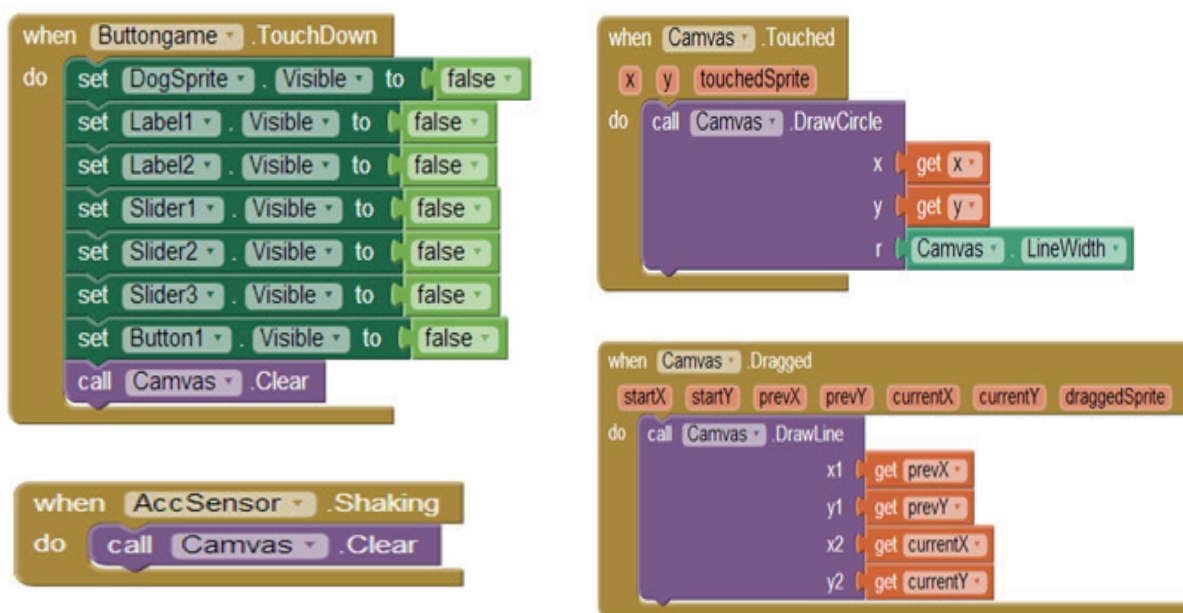
Πίνακας 7.1 Τα συστατικά μέρη της εφαρμογής και οι ιδιότητες τους			
Από την ομάδα του πλαισίου Palette	Επιλέγουμε το αντικείμενο και το μεταφέρουμε στο πλαίσιο Viewer	Αλλάζουμε το Όνομα με το κουμπί Rename	Μεταβάλλουμε τις ιδιότητες - Properties
	Screen1 (Είναι ήδη εγκατεστημένο, οπότε αλλάζουμε μόνο τις ιδιότητες)		BackgroundColor: LightGrey Screen Orientation: Portrait Scrollable: (No) Title:Κεντρικό μενού
Layout	Horizontal Arrangement	Horizontal Arrangement1	PhotoArrangement Width: Fill Parent AlignHorizontal: Center
User Interface	Button (το μεταφέρουμε μέσα στο πλαίσιο Horizontal Arrangement)	Buttonscreensaver	BackgroundColor: Cyan Shape: Rounded Text: Screensaver
User Interface	Button (το μεταφέρουμε μέσα στο πλαίσιο Horizontal Arrangement)	Buttongame	BackgroundColor: Green Shape: Rounded Text: Game
User Interface	Button (το μεταφέρουμε μέσα στο πλαίσιο Horizontal Arrangement)	ButtonMO	BackgroundColor: Red Shape: Rounded Text: Μέσος Όρος
User Interface	Label	Label1	Visible: Hidden
User Interface	Slider	Slider1	MaxValue: 20.0 MinValue: 1.0 ThumbPosition: 5 Visible: hidden Width: Fill parent
User Interface	Slider	Slider2	MaxValue: 20.0 MinValue: 1.0 ThumbPosition: 10 Visible: hidden Width: Fill parent
User Interface	Slider	Slider3	MaxValue: 20.0 MinValue: 1.0 ThumbPosition: 15 Visible: hidden Width: Fill parent
User Interface	Label	Label2	Visible: Hidden
User Interface	Button	Button1	Text: Τελικό αποτέλεσμα TextAlignment: center Visible: hidden Width: Fill parent
Drawing and Animation	Canvas	Canvas	Paint Color: Blue Width: Fill Parent Height: Fill Parent
Sensors	Accelerometer Sensor	AccSensor	
Drawing and Animation	ImageSprite	DogSprite	Interval: 10 Picture: SleepyDog.jpg Rotates: (No) Visible: (Yes)
Media	Sound	DogSound	Source: Bark.mp3 MinimumInterval: 300

Στη συνέχεια επιλέγουμε από πάνω δεξιά το κουμπί Blocks και μεταφερόμαστε στο περιβάλλον εργασίας όπου προγραμματίζουμε (App Inventor Blocks Editor). Δημιουργούμε τα παρακάτω σενάρια (blocks εντολών). Συγκεκριμένα, για να προγραμματίσουμε για ένα αντικείμενο, το επιλέγουμε από το πλαίσιο Blocks και από το συρτάρι εντολών που ανοίγει επιλέγουμε την εντολή και τη μεταφέρουμε στο πλαίσιο Viewer. Το περιβάλλον μάς βοηθάει να αποφύγουμε συντακτικά λάθη, μιας και σε αυτή την περίπτωση δεν «κουμπώνουν» οι εντολές μεταξύ τους.

(Α) Δημιουργούμε τα παρακάτω σενάρια εντολών για την επιλογή προστασίας της οθόνης.



(Β) Δημιουργούμε τα παρακάτω σενάρια για την επιλογή του παιχνιδιού σχεδίασης. Όπου ακουμπάει ο χρήστης ζωγραφίζει μια κουκκίδα και, όταν σύρει το δάκτυλο, ζωγραφίζει γραμμή.



Για να μπορούμε όμως να σχεδιάσουμε κάτι άλλο από την αρχή, θα πρέπει να καθαρίσουμε την οθόνη. Αυτό γίνεται, αν κουνήσουμε τη συσκευή.

(Γ) Δημιουργούμε τα παρακάτω σενάρια για την επιλογή του υπολογισμού του Μέσου Όρου (ΜΟ) και των αποτελεσμάτων προαγωγής του μαθητή σε ένα μάθημα.

Αντιστοίχιση των μεταβλητών

Βαθμός Α τετραμήνου - A
Βαθμός Β τετραμήνου - B
Βαθμός γραπτού - G
Μέσος όρος μαθήματος - mo

```
initialize global A to 0
initialize global B to 0
initialize global G to 0
initialize global mo to 0
```

Δημιουργία και αρχικοποίηση των μεταβλητών A, B, G, mo.

```
when ButtonMO.TouchDown
do
  set DogSprite.Visible to false
  call Canvas.Clear
  set Label1.Visible to true
  set Slider1.Visible to true
  set Slider2.Visible to true
  set Slider3.Visible to true
  set Button1.Visible to true
  set Label1.Text to join
    "Εισάγετε τους βαθμούς των τετραμήνων "
    "και το γραπτό των εξετάσεων "
```

Οι ενέργειες που κάνουμε, όταν πατήσουμε το κουμπί «Μέσος Όρος», αφορούν κυρίως στον καθαρισμό της οθόνης και στην εμφάνιση των ετικετών και του κουμπιού για τον υπολογισμό του μέσου όρου του μαθήματος.

```
when Slider1.PositionChanged
  thumbPosition
do
  set global A to get thumbPosition
  set Label1.Text to get global A

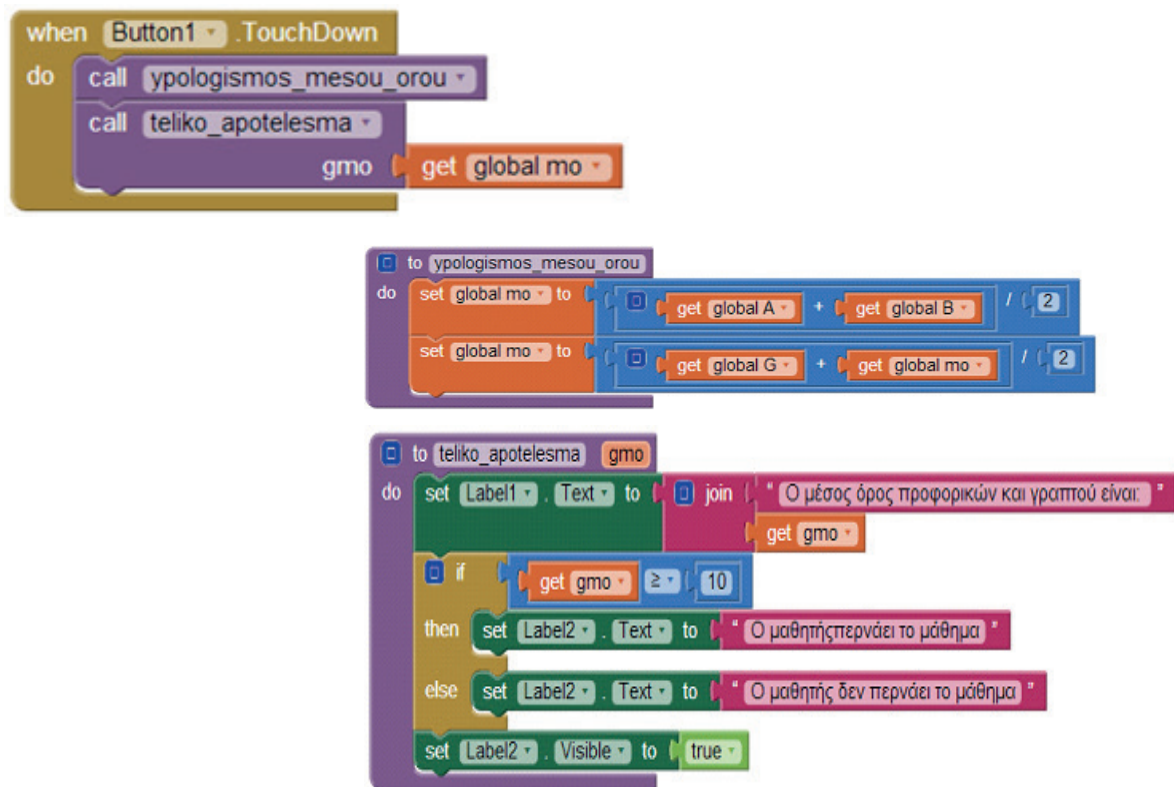
when Slider2.PositionChanged
  thumbPosition
do
  set global B to get thumbPosition
  set Label1.Text to get global B

when Slider3.PositionChanged
  thumbPosition
do
  set global G to get thumbPosition
  set Label1.Text to get global G
```

Με τον διπλανό κώδικα επιτυγχάνουμε, μετακινώντας την μπάρα πάνω σε μια κλίμακα από 1 έως 20, να ενημερώνονται οι αντίστοιχες μεταβλητές που αφορούν στους 3 βαθμούς του μαθήματος:

Slider1 για τη μεταβλητή A
Slider2 για τη μεταβλητή B
Slider3 για τη μεταβλητή G

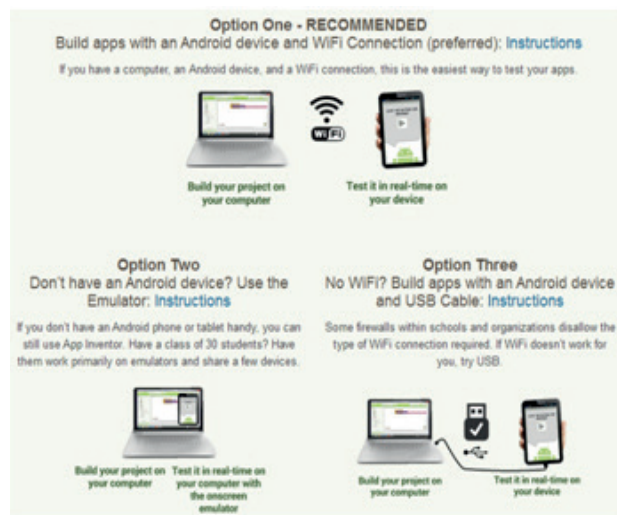
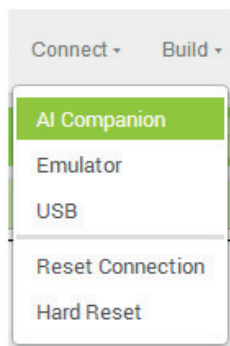
Όταν πατήσουμε το κουμπί «Τελικό αποτέλεσμα», τότε καλούμε τις Procedures (διαδικασίες) για τον υπολογισμό του μέσου όρου του μαθήματος και για την εμφάνιση του τελικού αποτελέσματος προαγωγής ή απόρριψης του μαθητή στο μάθημα. Αξίζει να σημειωθεί ότι κατά την κλήση της διαδικασίας «teliko_apotelesma» περνάμε ως όρισμα τη μεταβλητή mo, η οποία μεταβιβάζει την τιμή της στην gmo.



Φάση 4η: Λειτουργία

Οποιαδήποτε στιγμή μπορούμε να κάνουμε έλεγχο της λειτουργίας της εφαρμογής μας, για να εντοπίσουμε πιθανά λάθη και να επιστρέψουμε στη φάση της υλοποίησης ή ακόμη και της σχεδίασης για διορθώσεις ή και βελτιώσεις.

Επιλέγουμε από το μενού **Connect** όποια από τις επιλογές μπορούμε να δοκιμάσουμε. Όπως φαίνεται και στην εικόνα 7.12, αν επιλέξουμε: 1) **AI Companion**, μας δίνεται η δυνατότητα να εκτελέσουμε τον κώδικα στη φορητή συσκευή με WiFi σύνδεση, 2) **Emulator**, βλέπουμε τα αποτελέσματα της εκτέλεσης με τη χρήση προσομοιωτή στην οθόνη του υπολογιστή και 3) **USB**, μας δίνεται η δυνατότητα να εκτελέσουμε τον κώδικα στη φορητή συσκευή με ενσύρματη σύνδεση μέσω USB θύρας.



Εικόνα 7.12. Οι τρόποι σύνδεσης για την εκτέλεση της εφαρμογής σε φορητές συσκευές ή προσομοίωση στον υπολογιστή (<http://appinventor.mit.edu/explore/ai2/setup.html>)

Φάση 5η: Συντήρηση

Κατά τη φάση αυτή είναι δυνατό να γίνουν προτάσεις και από τους χρήστες για αλλαγές και βελτιώσεις της εφαρμογής. Για παράδειγμα, ορισμένες προτάσεις θα μπορούσαν να είναι:

1. Να κινείται ο σκύλος επάνω στην οθόνη.
2. Να γίνεται λήψη μιας φωτογραφίας από την κάμερα του κινητού και ο χρήστης να ζωγραφίζει πάνω στην εικόνα αυτή.
3. Να προστεθούν δυνατότητες αποθήκευσης και εκτύπωσης της ζωγραφιάς.
4. Να έχουμε δυνατότητα επιλογής χρώματος.
5. Να δημιουργηθεί μια λίστα με τα μαθήματα της τάξης, ώστε να φαίνεται και ο τίτλος του μαθήματος.
6. Να ενημερώσουμε την εφαρμογή σε οποιαδήποτε περίπτωση αλλαγής του τρόπου υπολογισμού του μέσου όρου.



Εικόνα 7.13. Διεπαφή χρήσης της εφαρμογής

Ερωτήσεις - Δραστηριότητες:

1. Επεκτείνετε και εμπλουτίστε την εφαρμογή, υλοποιώντας τις προτάσεις που αναφέρονται στη φάση της συντήρησης.
2. Ποια εφαρμογή πιστεύετε ότι θα μπορούσατε να φτιάξετε με την ομάδα σας; Συμβουλευτείτε τον καθηγητή σας και υλοποιήστε τη.

7.2 Αντικειμενοστρεφής προγραμματισμός σε 3D περιβάλλον**Αντικειμενοστρεφής προγραμματισμός**

Με το πέρασμα των χρόνων τα προγράμματα γίνονται μεγαλύτερα σε μέγεθος και πιο πολύπλοκα σε δομή και λειτουργίες. Επίσης, ο προσδιορισμός των απαιτήσεων και η συντήρηση του λογισμικού δυσκολεύει. Ο **αντικειμενοστρεφής προγραμματισμός (object-oriented programming)** αποτελεί μια διαδεδομένη προσέγγιση για δημιουργία προγραμμάτων, η οποία προσφέρει καλύτερη αντιμετώπιση των παραπάνω προβλημάτων. Αντικειμενοστρεφής είναι ο χαρακτηρισμός που σημαίνει «στραμμένος (προσανατολισμένος) σε αντικείμενα». Όταν κάποιος προγραμματίζει με αντικειμενοστρεφή τρόπο, διασπά ένα πρόβλημα στα συστατικά του στοιχεία. Κάθε στοιχείο μετατρέπεται σε ένα αυτοτελές **αντικείμενο (object)**, το οποίο περιέχει τις δικές του εντολές και τα δεδομένα που σχετίζονται με αυτό το αντικείμενο. Με αυτή τη διαδικασία μειώνεται η πολυπλοκότητα και γίνεται ευκολότερος ο χειρισμός των μεγάλων προγραμμάτων.

Μια **κλάση (class)** είναι ένα πρότυπο (καλούπι) που χρησιμοποιείται για τη δημιουργία ενός αντικείμενου. Κάθε αντικείμενο που δημιουργείται από την ίδια κλάση έχει παρόμοια, αν όχι ίδια, χαρακτηριστικά. Ένα αντικείμενο αποτελεί ένα μοναδικό και συγκεκριμένο **στιγμιότυπο (instance)** της κλάσης στην οποία

**Κληρονομικότητα:**

η διεργασία μέσω της οποίας μια κλάση μπορεί να αποκτήσει (κληρονομήσει) τις ιδιότητες και μεθόδους μιας άλλης κλάσης. Έτσι δημιουργείται μια ιεραρχική ταξινόμηση. Π.χ. κλάση **Φρούτο**, υποκλάση **Μήλο** και υποκλάση **Φιρίκι** (Ελληνική ποικιλία). Επειδή το φιρίκι έχει κληρονομήσει όλα τα ποιοτικά χαρακτηριστικά των φρούτων, χρειάζεται να ορίσουμε γι' αυτό μόνο τα χαρακτηριστικά που το κάνουν μοναδικό. Άλλο παράδειγμα, κλάση **Μέσο μεταφοράς**, υποκλάση **Όχημα**, υποκλάση **Αυτοκίνητο**.



Δημοφιλείς γλώσσες αντικειμενοστρεφούς προγραμματισμού είναι η Java, η C++ και η Python.

ανήκει. Πρώτα ορίζονται οι κλάσεις και μετά δημιουργούνται τα αντικείμενα. Τα χαρακτηριστικά μιας κλάσης αντικειμένων ονομάζονται **ιδιότητες (properties)** και οι διαδικασίες που ορίζουν τις συμπεριφορές της ονομάζονται **μέθοδοι (methods)**. Οι μέθοδοι στις οποίες εκτελούνται μόνο εντολές και δεν επιστρέφεται κάποια τιμή ονομάζονται **διαδικασίες (procedures)**, ενώ οι μέθοδοι στις οποίες επιστρέφεται κάποια τιμή ονομάζονται **συναρτήσεις (functions)**. Για πα-

Κλάση: ρομπότ

Ιδιότητες:

θερμοκρασία, θέση, ταχύτητα

Μέθοδοι:

- εκκίνηση εξερεύνησης
- έλεγχος τρέχουσας θερμοκρασίας
- αναφορά τρέχουσας θέσης
- ορισμός ταχύτητας

ράδειγμα, σε ένα πρόγραμμα προσομοίωσης ρομποτικών συσκευών εξερεύνησης μπορούμε να ορίσουμε ως κλάση το «ρομπότ» και στη συνέχεια να δημιουργήσουμε αντικείμενα ρομπότ για διάφορες μορφές εξερεύνησης. (π.χ. ρο-

μπότ εξερεύνησης βυθού, ρομπότ εξερεύνησης ηφαιστείου).

Το περιβάλλον προγραμματισμού Alice



Στο επίσημο site του Alice <http://www.alice.org> μπορείτε να «κατεβάσετε» το αρχείο εγκατάστασής του, να βρείτε οδηγούς εκμάθησης, ομάδες συζητήσεων, δημοσιεύσεις κ.ά. Το Alice διατίθεται δωρεάν, αναπτύσσεται στη Java από το Πανεπιστήμιο Carnegie Mellon και στηρίζεται οικονομικά από σημαντικές εταιρείες του χώρου της Πληροφορικής όπως οι Oracle, Electronic Arts, Sun Microsystems, Intel και Microsoft.

Το Alice είναι ένα ελεύθερα διαθέσιμο και καινοτόμο 3D (τρισεδιάστατο) περιβάλλον προγραμματισμού που καθιστά εύκολη τη δημιουργία κινούμενων γραφικών (animation) για την αφήγηση μιας ιστορίας, την ανάπτυξη διαδραστικών παιχνιδιών ή τη δημιουργία βίντεο που μπορεί να διαμοιραστεί στο Διαδίκτυο. Ακολουθεί την αντικειμενοστρεφή προσέγγιση προγραμματισμού. Στο Alice, 3D αντικείμενα (π.χ. σκηνικά, άνθρωποι, ζώα, φυτά, οχήματα) σχηματίζουν έναν εικονικό κόσμο και ο προγραμματιστής δημιουργεί οπτικά ένα πρόγραμμα με σύρσιμο και ταίριασμα κατάλληλων πλακιδίων (tiles ή blocks) για τον ορισμό των ιδιοτήτων, των συμπεριφορών και των αλληλεπιδράσεων των παραπάνω αντικειμένων. Τα αντικείμενα αποτελούν στιγμιότυπα κλάσεων που οργανώνονται με σχέσεις ιεραρχίας μεταξύ τους και στα οποία ισχύουν οι αρχές της κληρονομικότητας. Επίσης, στο Alice έχουμε **προγραμματισμό οδηγούμενο από γεγονότα (event-driven programming)**. Κάθε φορά που ο χρήστης κάνει κλικ με το ποντίκι ή πατάει ένα πλήκτρο, δημιουργείται ένα γεγονός που προκαλεί μια απάντηση. Για παράδειγμα, αν κάνουμε «κλικ σε ένα όχημα» (γεγονός), αυτό «αρχίζει να κινείται» (απάντηση). Ο χειρισμός των γεγονότων γίνεται με κατάλληλες μεθόδους.

Διαδικασία δημιουργίας μιας εφαρμογής στο Alice.

Για να δημιουργήσουμε μια εφαρμογή στο Alice, ακολουθούμε τα παρακάτω βήματα:

1. Ενεργοποιούμε το Alice3, το οποίο έχουμε κατεβάσει από

τον ιστότοπο <http://www.alice.org/> και το έχουμε εγκαταστήσει τοπικά στον υπολογιστή μας.

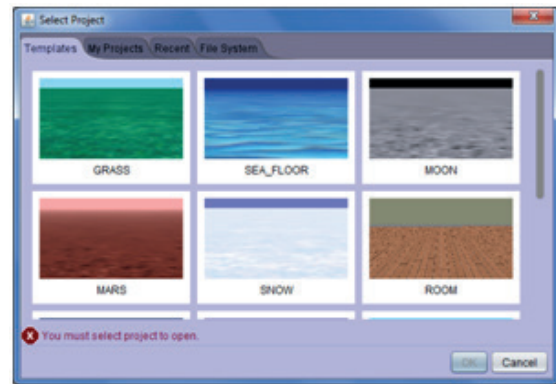
2. Στο πλαίσιο διαλόγου **Select Project** (εικόνα 7.14) επιλέγουμε την αρχική σκηνή (**Templates**) του ειδικτικού μας κόσμου (επιφάνεια και ατμόσφαιρα).
3. Στην οθόνη μας εμφανίζεται το παράθυρο της εικόνας 7.15 όπου προγραμματίζουμε. Χωρίζεται σε 4 μέρη:

- ✓ **χώρος Α: Σκηνή (Scene)**, όπου αναπαριστάται ο εικονικός κόσμος που σχεδιάζουμε.
- ✓ **χώρος Β: Συντάκτης Κώδικα (Code Editor)**, όπου προγραμματίζουμε μεταφέροντας τις μεθόδους από τους χώρους Γ και Δ, αφού πρώτα επιλέξουμε τη σωστή καρτέλα (περιοχή B1) στην οποία συντάσσουμε τον κώδικα.
- ✓ **χώρος Γ: Μέθοδοι (Procedures, Functions)**. Τις χρησιμοποιούμε, για να αποκτήσει συμπεριφορές το εκάστοτε αντικείμενο που έχουμε επιλέξει στο Γ1.
- ✓ **χώρος Δ: Μέθοδοι, Έλεγχος (Control)**. Τις χρησιμοποιούμε, όταν θέλουμε να ομαδοποιήσουμε έναν αριθμό μεθόδων βάσει κάποιου ελέγχου ή συνθήκης (π.χ. if_, while_).

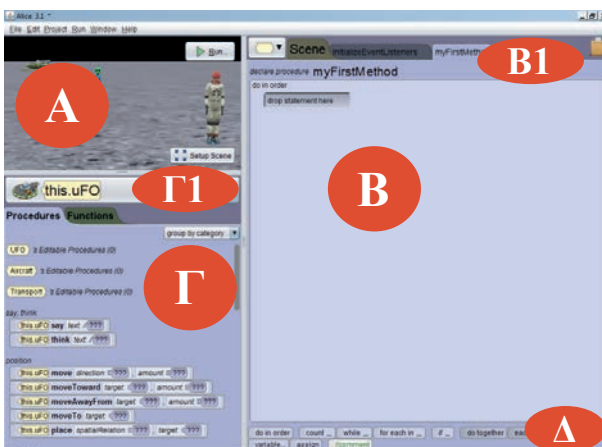
4. Αρχικά όμως πρέπει να σχεδιάσουμε τον εικονικό μας κόσμο, ώστε να μπορούμε στη συνέχεια να τον προγραμματίσουμε. Επιλέγουμε στη σκηνή A το κουμπί **Setup Scene** και μεταφερόμαστε στο αντίστοιχο παράθυρο (εικόνα 7.16). Χωρίζεται σε 3 μέρη και η αρίθμηση με την οποία αναφέρονται είναι και η σειρά που ακολουθούμε, για να σχεδιάσουμε τον κόσμο μας:

- i. Αρχικά από τον χώρο X επιλέγουμε αντικείμενα και τα τοποθετούμε στη σκηνή (Ψ). Τα αντικείμενα είναι ομαδοποιημένα σε κατηγορίες - κλάσεις (Biped-δίποδα, Flyer-ιπτάμενα, Prop-υποστηρικτικά, Quadruped-τετράποδα, Swimmer-υποθαλάσσια και Transport-μέσα μεταφοράς). Για ορισμένα αντικείμενα (άνθρωποι) έχουμε τη δυνατότητα να επιλέξουμε την ενδυμασία τους.
- ii. Τοποθετούμε τα αντικείμενα στην επιθυμητή θέση στη σκηνή Ψ.
- iii. Ρυθμίζουμε τις αρχικές ιδιότητες του κάθε αντικειμένου στο χώρο Z.

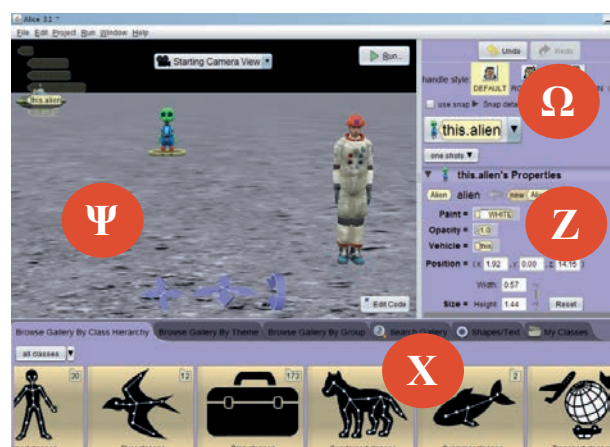
5. Για να επιστρέψουμε και να προγραμματίσουμε, επιλέγουμε το κουμπί **Edit Code** από τη σκηνή Ψ.
6. Οποιαδήποτε στιγμή επιθυμούμε να δούμε τα αποτελέσματα της εκτέλεσης του κώδικα επιλέγουμε από τη σκηνή A το κουμπί **Run**.



Εικόνα 7.14. Επιλογή σκηνής του εικονικού κόσμου



Εικόνα 7.15. Παράθυρο ανάπτυξης του προγράμματος



Εικόνα 7.16. Παράθυρο σχεδίασης του εικονικού κόσμου

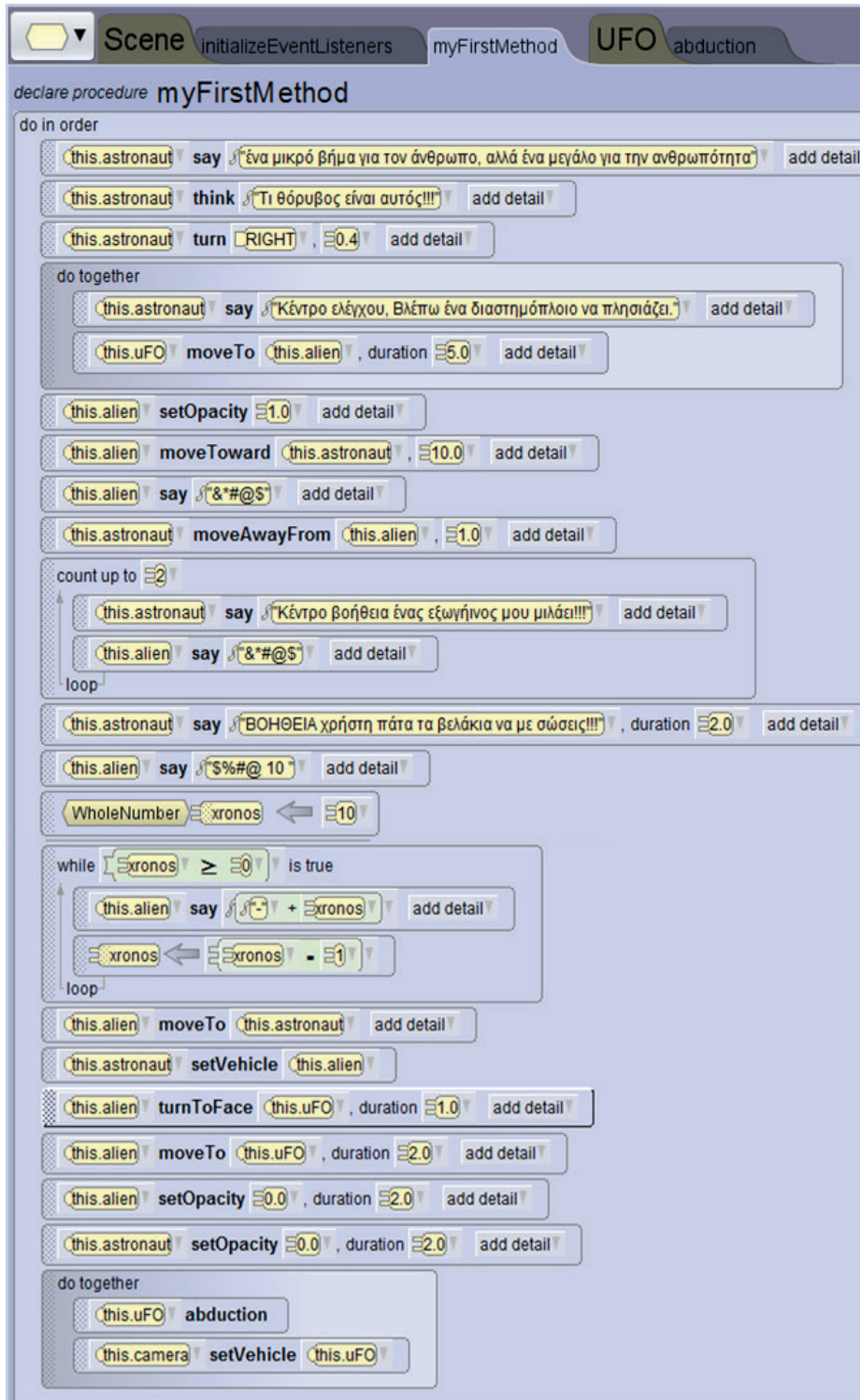
Project με την εφαρμογή Alice.

Στη συνέχεια θα δημιουργήσουμε μια ολοκληρωμένη εφαρμογή. Η περιγραφή του σεναρίου έχει ως εξής. Υπάρχει ένας αστροναύτης (**AdultPerson**) στη Σελήνη, αντιλαμβάνεται ότι πλησιάζει ένα σκάφος και στρέφεται προς εκείνη την κατεύθυνση. Όμως το σκάφος είναι Άγνωστης Ταυτότητας Ιπτάμενο Αντικείμενο (**UFO**). Γίνεται η προσσελήνωση, κατεβαίνει ένας εξωγήινος (**Alien**) και πλησιάζει τον αστροναύτη. Ο αστροναύτης καλεί σε βοήθεια το κέντρο ελέγχου και ζητάει από τον χρήστη να τον απομακρύνει από τον Alien. Αφού δεν έχει επιτευχθεί ο επιθυμητός διάλογος μεταξύ τους, ο εξωγήινος απάγει τον αστροναύτη στο σκάφος και αναχωρούν για τον πλανήτη του.

Τα βήματα που πρέπει να ακολουθήσουμε είναι τα παρακάτω:

1. Ενεργοποιούμε το Alice και από τα templates επιλέγουμε το **Moon** (εικόνα 7.14). Για να μην ξεχάσουμε να αποθηκεύσουμε την εργασία μας, επιλέγουμε μενού **File**→**Save As** και για κάθε επόμενη φορά το **File**→**Save**.
2. Μεταφερόμαστε στο παράθυρο σχεδίασης του κόσμου, πατώντας το κουμπί **Setup Scene**. Εισάγουμε τον αστροναύτη από την **κλάση Biped**, χρησιμοποιώντας τη μέθοδο **new Adult(...)**. Αφού του φορέσουμε τη στολή, τον τοποθετούμε στη σκηνή Ψ μπροστά και δεξιά. Τον μετονομάζουμε σε **astronaut** (εικόνα 7.16).
3. Εισάγουμε τον εξωγήινο από την **κλάση Biped**, χρησιμοποιώντας την μέθοδο **new Alien()**. Τον τοποθετούμε στη μέση της σκηνής και ορίζουμε την ιδιότητα **Opacity** (ορατότητα) σε 0.0. Αρχικά δεν φαίνεται ο alien, για να μπορέσουμε να τον εμφανίσουμε αργότερα και να δημιουργείται η εντύπωση ότι βγαίνει από το UFO.
4. Εισάγουμε το UFO από την **κλάση Transport** και την υποκλάση **Aircraft**, χρησιμοποιώντας τη μέθοδο **new UFO()**. Τον τοποθετούμε στη σκηνή Ψ πίσω και αριστερά.
5. Μπορούμε να μετακινήσουμε, περιστρέψουμε, ανυψώσουμε, αλλάξουμε μέγεθος κ.ά. στα αντικείμενά μας με τις επιλογές **Default**, **Rotation**, **Translation**, **Resize** (Ω, εικόνα 7.16), καθώς επίσης και με τα μπλε βελάκια που βρίσκονται μπροστά και στο κέντρο της σκηνής Ψ.
6. Για να επιστρέφουμε στο αρχικό παράθυρο, πατάμε το κουμπί **Edit Code**.
7. Συντάσσουμε τον κώδικα της εικόνας 7.17 στη δική μας μέθοδο **MyFirstMethod** (B1) Να σημειώσουμε ότι μετά από το σύρσιμο μιας μεθόδου στον συντάκτη κώδικα τροποποιούμε τις παραμέτρους της. Επιπλέον δυνατότητες ορισμού παραμέτρων μάς δίνει η επιλογή του **add detail**.
8. Για να δημιουργήσουμε μια procedure, επιλέγουμε το εξάγωνο από την περιοχή B1→UFO →**Add UFO Procedure**. Πληκτρολογούμε το όνομα της διαδικασίας **abduction**, οπότε δημιουργούνται 2 νέες καρτέλες (UFO, abduction). Εισάγουμε τις μεθόδους όπως φαίνονται στην εικόνα 7.18.
9. Για να δώσουμε τη δυνατότητα να χειρίζεται ο χρήστης με το δεξί και αριστερό βελάκι του πληκτρολογίου τον astronaut, επιλέγουμε καρτέλα **InitializeEventListeners**→ **Add Event Listeners**→ **Keyboard**→ **addArrowKeyPressListeners**. Στη συνέχεια μεταφέρουμε τη μέθοδο ελέγχου **if** από τον χώρο Δ στον συντάκτη κώδικα (B) και δημιουργούμε τον κώδικα όπως φαίνεται στην εικόνα 7.19. Η λογική που εφαρμόζουμε είναι «Αν (if) πατηθεί το αριστερό πλήκτρο, τότε (then) να κινηθεί ο astronaut αριστερά, αλλιώς, αν (if) πατηθεί το δεξί βελάκι, τότε (then) να κινηθεί ο astronaut δεξιά, αλλιώς (else) να εκφράσει τη δυσaréσκειά του. Ο λόγος που επιλέξαμε για να υλοποιηθεί αυτός ο κώδικας σε αυτή την καρτέλα και όχι εντός της μεθόδου **MyFirstMethod** είναι ότι ο κώδικας που γράφεται σε αυτό τη σημείο εκτελείται από την αρχή εκτέλεσης της εφαρμογής μέχρι το τέλος αυτής.

10. Οποιαδήποτε στιγμή μπορούμε να εκτελέσουμε τον κώδικα και να ελέγξουμε τα αποτελέσματα που εμφανίζονται με τη μορφή βίντεο.
11. Δεν ξεχνάμε κατά διαστήματα και στο τέλος να αποθηκεύσουμε την εφαρμογή μας.



Πίνακας 7.2 Επεξήγηση εντολών στην MyFirstMethod

Ο astronaut «μιλάει» και «σκέφτεται» το αντίστοιχο κείμενο που είναι γραμμένο.

Στρίβει προς μια κατεύθυνση. Το 1.0 είναι μια πλήρης περιστροφή 360°.

Οι μέθοδοι say και moveTo μέσα στο do together εκτελούνται ταυτόχρονα. Η διάρκεια (duration) είναι σε δευτερόλεπτα.

Εμφανίζεται ο alien, μετακινείται μπροστά στον astronaut και του μιλάει, ενώ αυτός απομακρύνεται προς τα πίσω.

Επανάληψη του διαλόγου 2 φορές (δομή count up to).

Δίνεται χρονικό περιθώριο στον astronaut να συνετιστεί.

Μετά την κλήση για βοήθεια ορίζουμε μια μεταβλητή χρόνος με αρχική τιμή 10.

Στη δομή επανάληψης while ο alien αναφέρει τον εναπομείναντα χρόνο. Η μεταβλητή χρόνος μειώνεται κατά ένα σε κάθε επανάληψη, μέχρι να μηδενιστεί.

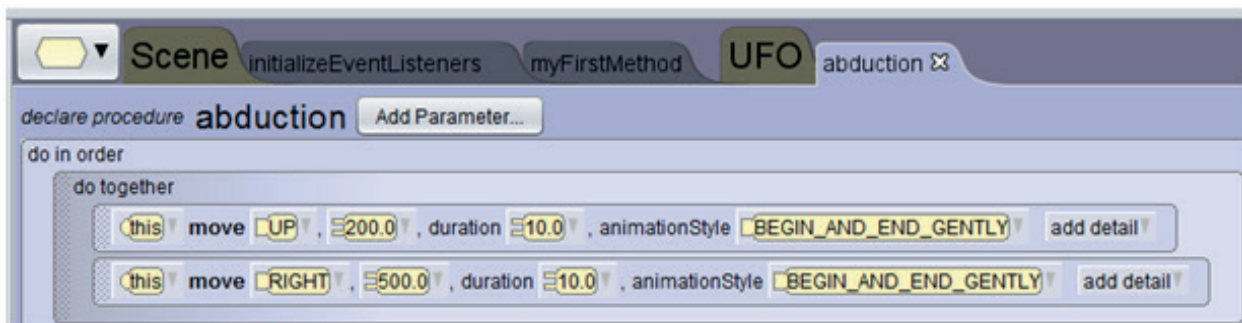
Οπότε και ο alien κινείται προς τον astronaut.

Με τη μέθοδο setVehicle τα δύο αντικείμενα αντιμετωπίζονται σαν ένα (ομαδοποίηση), δηλαδή η οποιαδήποτε αλλαγή στη συμπεριφορά του alien προκαλεί την ίδια συμπεριφορά και στον astronaut.

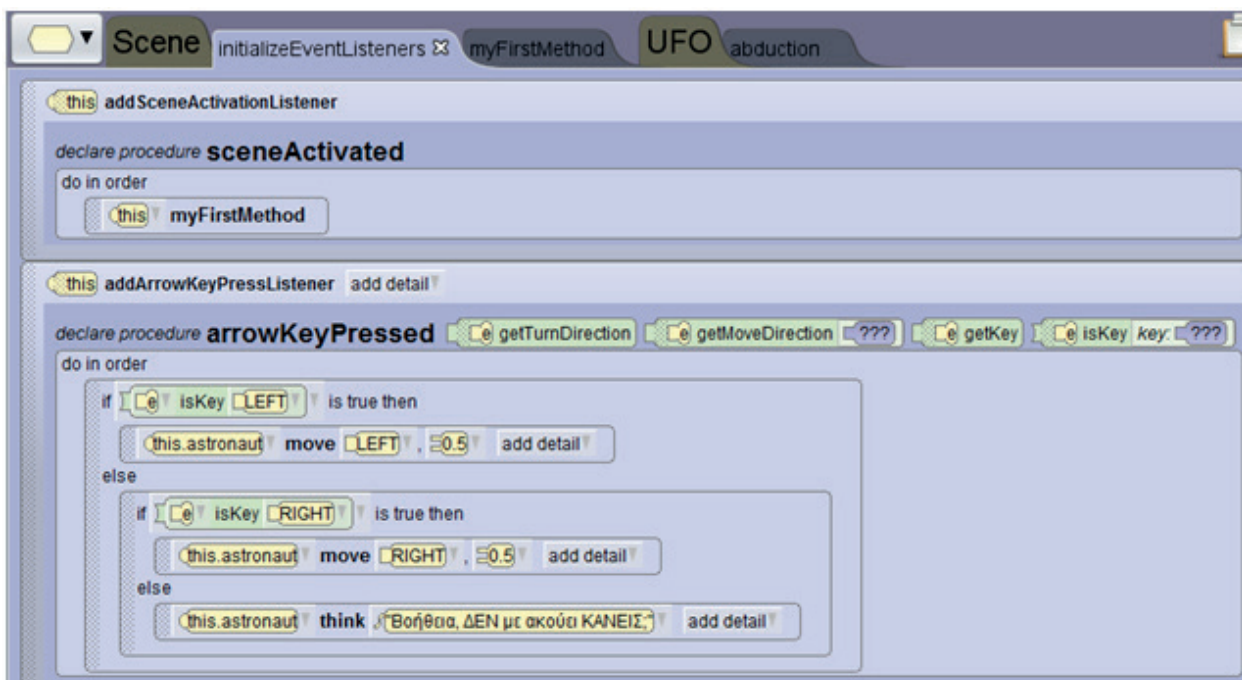
Εξαφανίζουμε (κάνουμε αόρατους) τους alien, astronaut.

Εκτελούνται παράλληλα: η κλήση της procedure: abduction (απαγωγή) και η ομαδοποίηση της κάμερας με το UFO.

Εικόνα 7.17. Ο κώδικας της μεθόδου MyFirstMethod



Εικόνα 7.18. Ο κώδικας της procedure-διαδικασίας που αφορά στην απαγωγή



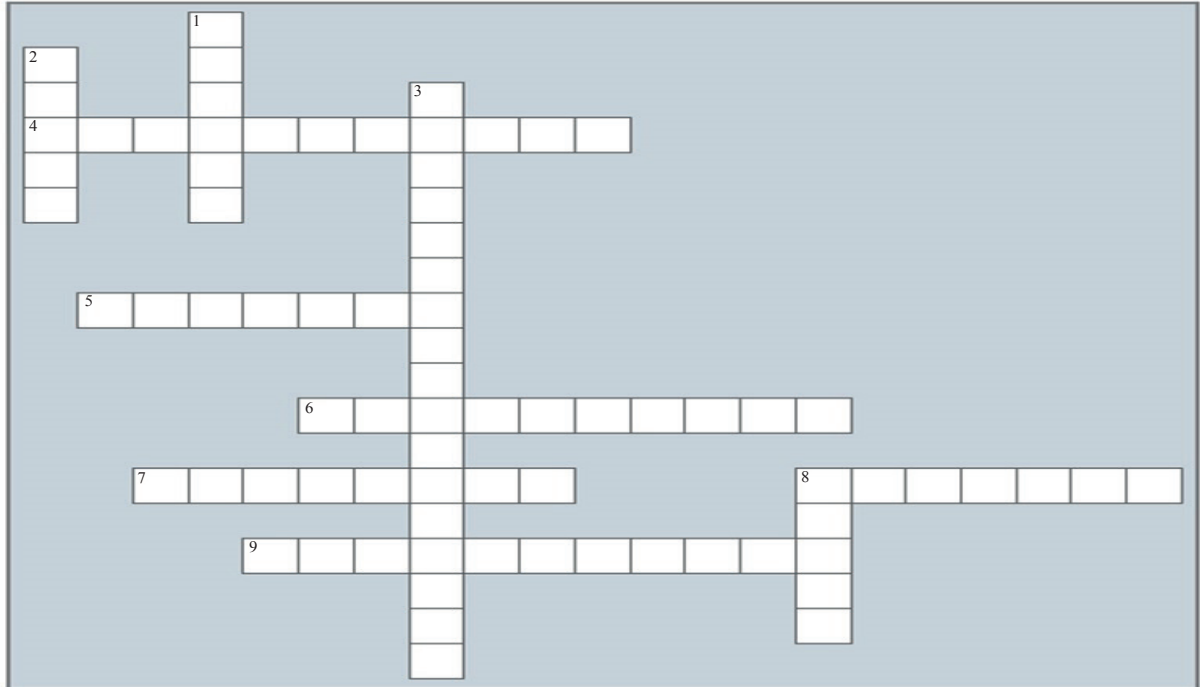
Εικόνα 7.19. Ο κώδικας στη μέθοδο InitializeEventListeners για τον χειρισμό του αστροναύτη από τον χρήστη

Ερωτήσεις - Δραστηριότητες:

1. Εμπλουτίστε την εφαρμογή με καλύτερες κινήσεις των αντικειμένων, όπως π.χ. όταν μετακινείται ο αστροναύτης δεξιά ή αριστερά να στρίβει και αναλόγως.
2. Επεκτείνετε την εφαρμογή, εισάγοντας έναν ακόμη αστροναύτη, και εμπλουτίστε την αλληλεπίδραση μεταξύ όλων των αντικειμένων.
3. Αφού πρώτα περιηγηθείτε σε έτοιμα προγράμματα που έχουν δημιουργηθεί με το Alice, συζητήστε σε ομάδες μια δική σας ιστορία που νομίζετε ότι είναι εφικτό να υλοποιηθεί στο συγκεκριμένο περιβάλλον. Συμβουλευτείτε τον καθηγητή σας και υλοποιήστε την.

Ασκήσεις Αυτοαξιολόγησης

1. Λύστε το σταυρόλεξο (συμπληρώστε το με κεφαλαία γράμματα της ελληνικής ή αγγλικής ορολογίας, κατά περίπτωση).



Οριζόντια

4. Οπτικό περιβάλλον προγραμματισμού με πλακίδια (blocks) που χρησιμοποιούμε για την ανάπτυξη εφαρμογών για φορητές συσκευές (έξυπνα κινητά, tablets) με λειτουργικό σύστημα Android.
5. Έτσι ονομάζονται οι διαδικασίες που ορίζουν τις συμπεριφορές των αντικειμένων στον αντικειμενοστρεφή προγραμματισμό.
6. Κατά τη φάση της _____ του κύκλου ζωής μιας εφαρμογής γίνονται όλες οι απαραίτητες προσαρμογές, αναβαθμίσεις και διορθώσεις της εφαρμογής, προκειμένου αυτή να συνεχίσει να χρησιμοποιείται απρόσκοπτα και αποδοτικά.
7. Κατά τη φάση της _____ του κύκλου ζωής μιας εφαρμογής καταγράφονται τα δεδομένα και τα ζητούμενα, οι προδιαγραφές και οι απαιτήσεις των μελλοντικών χρηστών της εφαρμογής.
8. Με το App Inventor αναπτύσσουμε εφαρμογές για φορητές συσκευές με Λειτουργικό σύστημα _____
9. Ορισμένοι προγραμματιστικοί _____ παρέχουν τρισδιάστατη (3D) απεικόνιση, π.χ. Kodu, Yenka.

Κατακόρυφα

1. Χαρακτηρίζεται έτσι το περιβάλλον προγραμματισμού, όπου ο προγραμματιστής δεν πληκτρολογεί εντολές, αλλά επιλέγει και τοποθετεί κατάλληλα γραφικά στοιχεία (πλακίδια –blocks) με τη διαδικασία “σύρε και άφησε”.
2. Είναι το πρότυπο, καλούπι που χρησιμοποιούμε, για να δημιουργήσουμε αντικείμενα στον αντικειμενοστρεφή προγραμματισμό.
3. Στο εκπαιδευτικό περιβάλλον Alice αναπτύσσουμε εφαρμογές με _____ προγραμματισμό.
8. 3D περιβάλλον προγραμματισμού για την ανάπτυξη εικονικών κόσμων με δυναμικές κινήσεις χαρακτήρων και αλληλεπίδραση με τον χρήστη.

2. Να χαρακτηρίσετε ως σωστή (Σ) ή λάθος (Λ) καθεμιά από τις παρακάτω προτάσεις:

Προτάσεις	Σ/Λ
1. Οι προγραμματιστικοί μικρόκοσμοι διαθέτουν ένα περιορισμένο ρεπερτόριο εντολών με απλή σύνταξη και απλές δομές δεδομένων, και διευκολύνουν τη δημιουργία παιχνιδιών με τη διαχείριση ενός κεντρικού ήρωα.	
2. Η γλώσσα Logo και τα Logo-like περιβάλλοντα ανήκουν στα επαγγελματικά προγραμματιστικά περιβάλλοντα.	
3. Τα συντακτικά λάθη στον προγραμματισμό εντοπίζονται μετά την εκτέλεση του προγράμματος, όταν παρατηρούμε ότι τα αποτελέσματα δεν είναι τα επιθυμητά.	
4. Στο προγραμματιστικό περιβάλλον App Inventor έχουμε τη δυνατότητα εκτέλεσης της εφαρμογής μας στον προσομοιωτή (Emulator) φορητής συσκευής.	
5. Το Alice είναι ένα ελεύθερα διαθέσιμο και δισδιάστατο (2D) περιβάλλον προγραμματισμού.	
6. Στο αντικειμενοστρεφές προγραμματιστικό περιβάλλον Alice έχουμε επίσης προγραμματισμό οδηγούμενο από γεγονότα.	

3. Αντιστοιχίστε τα δεδομένα μεταξύ των δύο στηλών. Η αντιστοίχιση είναι ένα προς πολλά (σε κάθε επιλογή της αριστερής στήλης αντιστοιχούν 2-3 από τη δεξιά στήλη).

(Α) Το περιβάλλον App Inventor υποστηρίζει	1- Σχεδίαση εφαρμογής
(Β) Μεταφραστικό πρόγραμμα	2- Διερμηνευτής
(Γ) Τρισδιάστατη απεικόνιση	3- Λεξιλόγιο
(Δ) Κάθε γλώσσα προγραμματισμού διαθέτει υποχρεωτικά	4- Υλοποίηση εφαρμογής
(Ε) Ο κύκλος ζωής εφαρμογών περιλαμβάνει	5- Δομή ελέγχου (if)
	6- Μεταγωγτιστής
	7- Kodu
	8- Δομή επανάληψης (While)
	9- Αλφάβητο
	10- Alice
	11- Συντακτικό
	12- Μεταβλητές (Variables)

Θέματα για Συζήτηση

1. Να συγκρίνετε τη διαδικασία κατασκευής ενός μεγάλου τεχνικού έργου (π.χ. μιας γέφυρας, ενός κτιρίου) με τα βήματα του κύκλου ζωής εφαρμογών και να καταγράψετε τις ομοιότητες και τις διαφορές.
2. Υποθέστε ότι θέλετε να αναπτύξετε μια εφαρμογή παιχνιδιού. Να περιγράψετε τις ενέργειες που θα κάνατε σε κάθε βήμα του κύκλου ζωής εφαρμογών.
3. Αφού ανατρέξετε στο κεφάλαιο 6 και παρατηρήσετε τις εικόνες 6.1, 6.2 και 6.3, να περιγράψετε ποια γλώσσα προγραμματισμού θα επιλέγατε, για να γράψετε το πρώτο σας πρόγραμμα.
4. Να καταγράψετε τις δυνατότητες και τα χαρακτηριστικά που θα θέλατε να έχει το προγραμματιστικό περιβάλλον που θα διαλέγατε, για να αναπτύξετε μια εφαρμογή.
5. Γιατί πιστεύετε ότι το Android είναι ένα δημοφιλές Λειτουργικό Σύστημα για φορητές συσκευές;
6. Θεωρείτε ότι είναι σημαντικό, όταν αναπτύσσετε μια εφαρμογή στο περιβάλλον App Inventor ή Alice, να ακολουθείτε τα βήματα του κύκλου ζωής εφαρμογών;